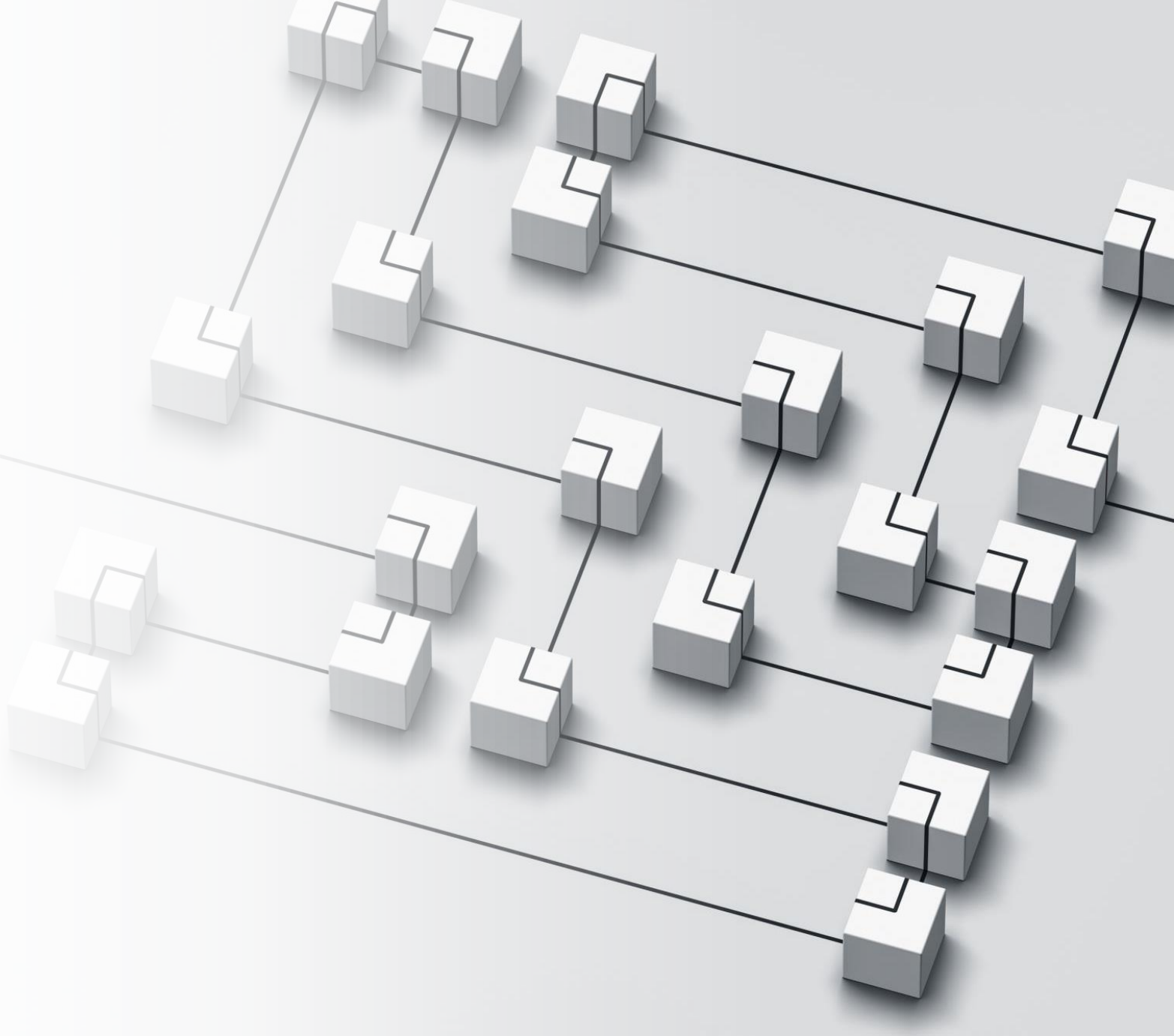
Anatomy of a Modular application Serivice Monitor

Java Platform Module System



Diposition

- Laying out a modular application's source code
- Creating module declarations
- Compiling modules
- Running a modular application

Service Monitor

The application's authors created four JARs:

- *observer*—Observes other services and checks availability
- *statistics*—Creates statistics from availability data
- *persistence*—Reads and writes statistics to the database with *hibernate*
- *monitor*—Triggers data collection and pipes the data through *statistics* into *persistence*; implements the REST API with *spark*

Service Monitor

Listing 2.1 `ServiceObserver` interface

```
public interface ServiceObserver {

 DiagnosticDataPoint gatherDataFromService();

}
```

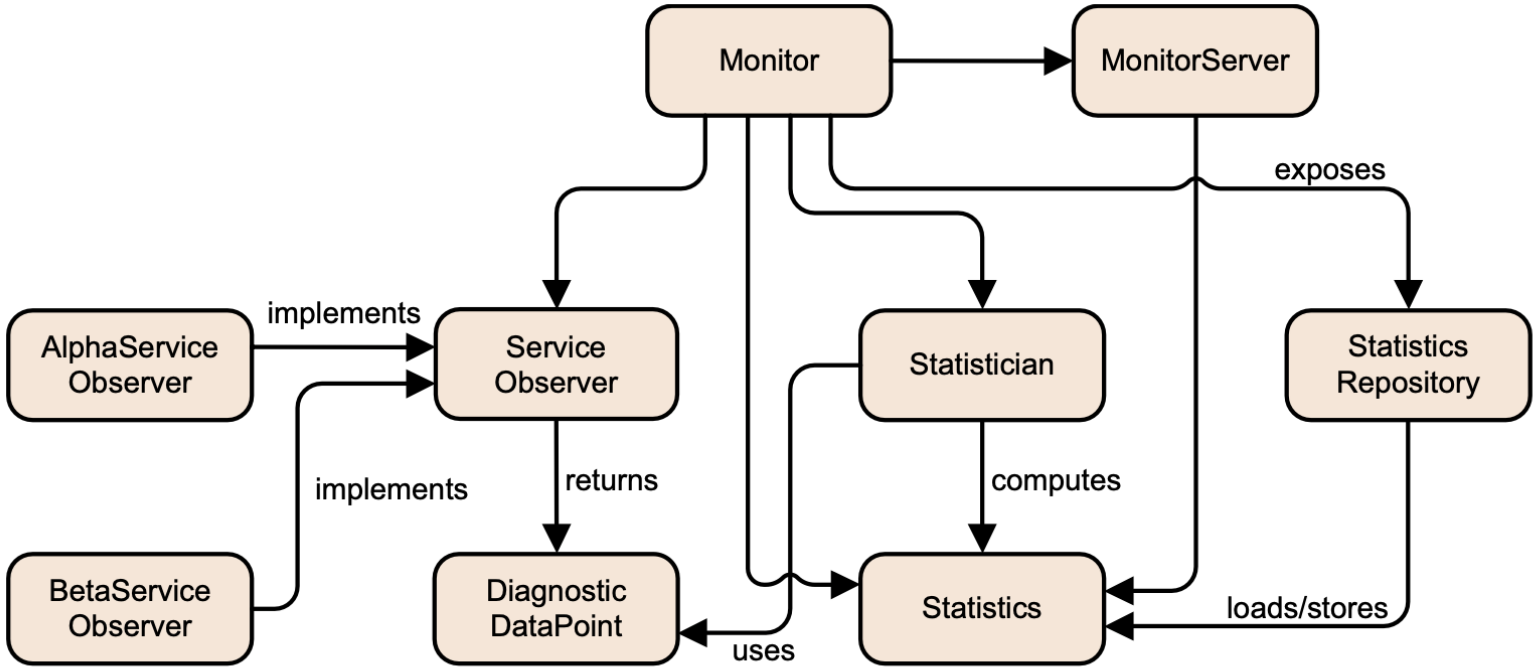


Figure 2.1 The classes making up the *ServiceMonitor* application. Two `ServiceObserver` implementations query services with Alpha and Beta APIs and return diagnostic data, which `Statistician` aggregates into `Statistics`. The statistics are stored and loaded by a repository as well as exposed via a REST API. `Monitor` orchestrates all this.

Listing 2.2 `Statistician` class

```
public class Statistician {

 public Statistics emptyStatistics() {
  return Statistics.empty();
 }

 public Statistics compute(
   Statistics currentStats,
   Iterable<DiagnosticDataPoint> dataPoints) {
  Statistics finalStats = currentStats;
  for (DiagnosticDataPoint dataPoint : dataPoints)
    finalStats = finalStats.merge(dataPoint);
  return finalStats;
 }

}
```

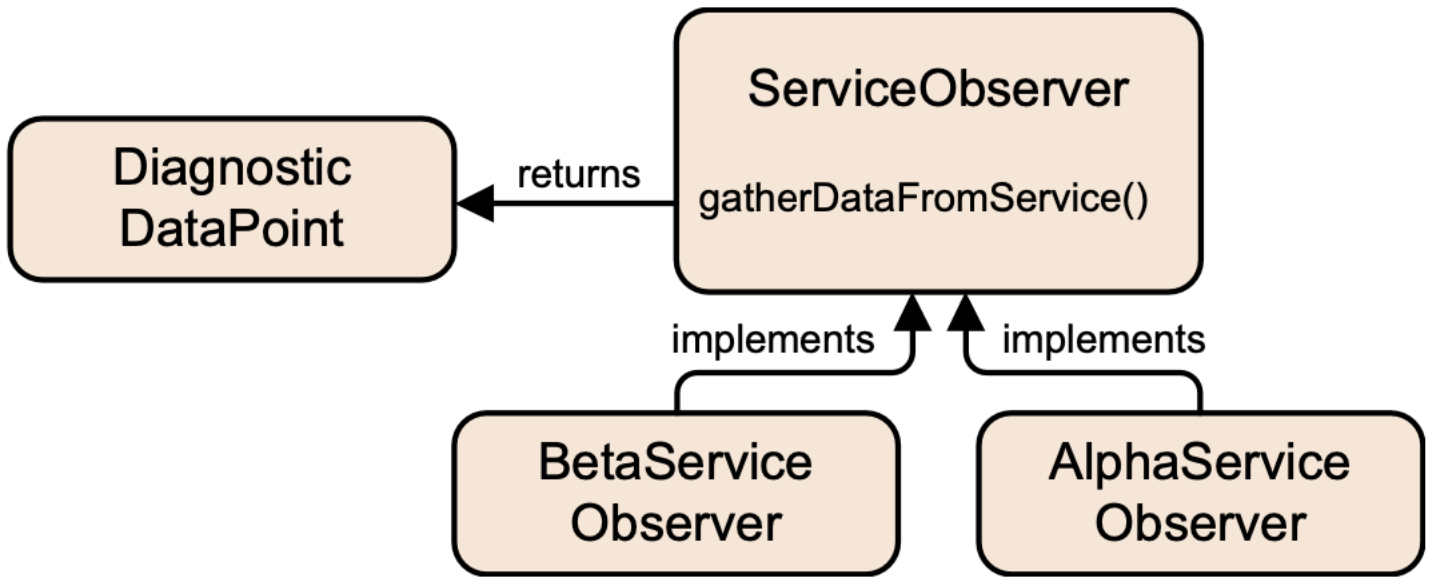


Figure 2.2 The observed services use two different API generations to expose the diagnostic data. Accordingly, the `ServiceObserver` interface has two implementations.

Listing 2.3 `StatisticsRepository` class

```
public class StatisticsRepository {

 public Optional<Statistics> load() { /* ... */ }

 public void store(Statistics statistics) { /* ... */ }

}
```

Listing 2.4 `Monitor` class and its `updateStatistics()` method

```
public class Monitor {

 private final List<ServiceObserver> serviceObservers;
 private final Statistician statistician;
 private final StatisticsRepository repository;
 private Statistics currentStatistics;
 // [...]

 private void updateStatistics() {
  List<DiagnosticDataPoint> newData = serviceObservers
   .stream()
   .map(ServiceObserver::gatherDataFromService)
   .collect(toList());
  Statistics newStatistics = statistician
   .compute(currentStatistics, newData);
  currentStatistics = newStatistics;
  repository.store(newStatistics);
 }

 // [...]
}
```

Listing 2.5 `MonitorServer` class

```
public class MonitorServer {

 private final Supplier<Statistics> statistics;

 public MonitorServer(Supplier<Statistics> statistics) {
  this.statistics = statistics;
 }

 // [...]

 private Statistics getStatistics() {
  return statistics.get();
 }

 // [...]

}
```

Types

All in all, you end up with these types:

- `DiagnosticDataPoint`—Availability data for a service in a time interval.
- `ServiceObserver`—Interface for service observation that returns `Diagnostic-DataPoint`.
- `AlphaServiceObserver` and `BetaServiceObserver`—Each observes a variant of services.
- `Statistician`—Computes statistics from `DiagnosticDataPoint`.
- `Statistics`—Holds the computed statistics.
- `StatisticsRepository`—Stores and retrieves statistics.
- `MonitorServer`—Answers REST calls for the statistics.
- `Monitor`—Ties everything together.

Cutting ServiceMonitor into modules

The most common way to modularize applications is by a separation of concerns. *ServiceMonitor* has the following, with the related types in parentheses:

- Collecting data from services (`ServiceObserver`, `DiagnosticDataPoint`)
- Aggregating data into statistics (`Statistician`, `Statistics`)
- Persisting statistics (`StatisticsRepository`)
- Exposing statistics via a REST API (`MonitorServer`)

But not only the domain logic generates requirements. There are also technical ones:

- Data collection must be hidden behind an API.
- The Alpha and Beta services each require a separate implementation of that API (`AlphaServiceObserver` and `BetaServiceObserver`).
- All concerns must be orchestrated (`Monitor`).

Structure

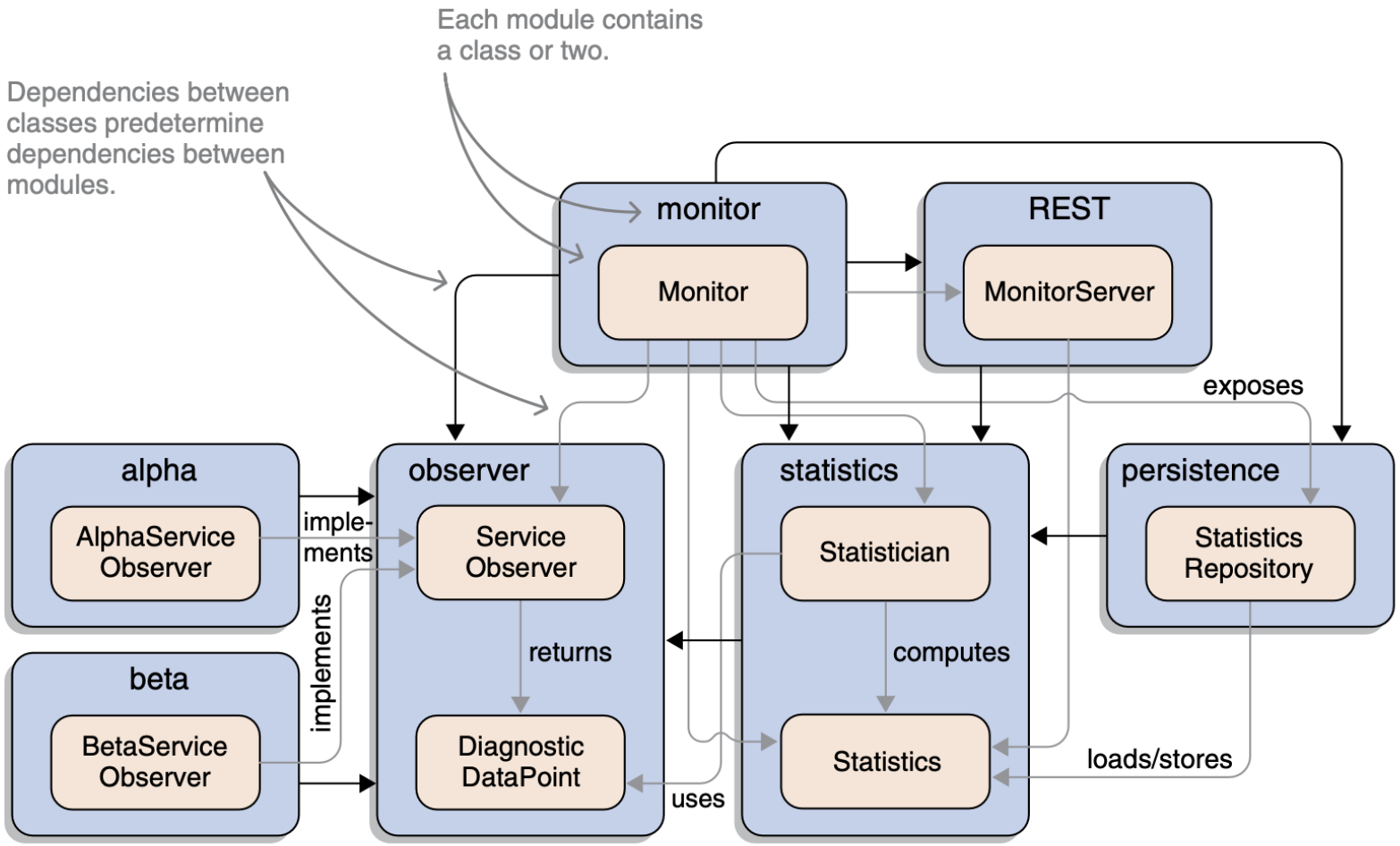
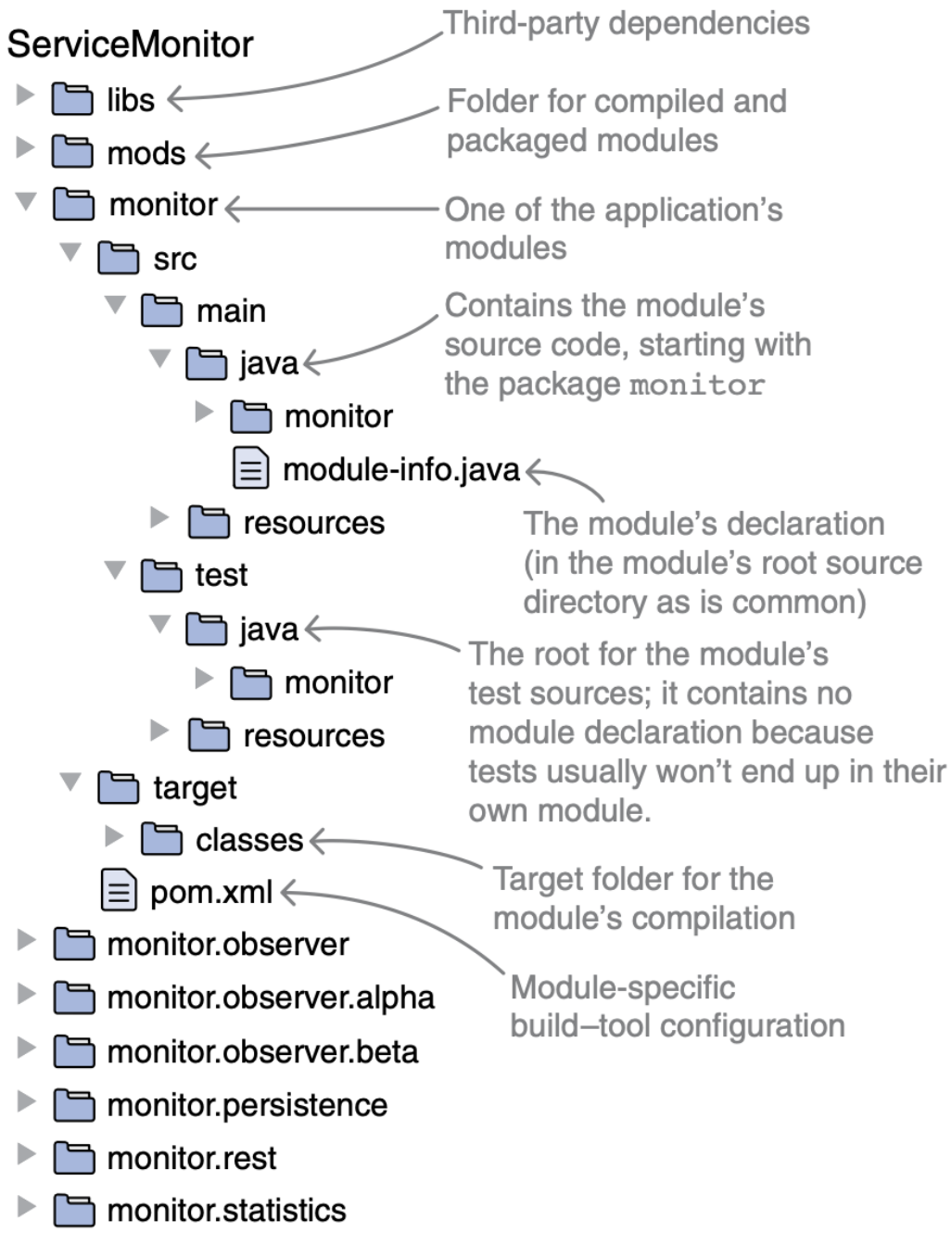


Figure 2.3 The *ServiceMonitor* application's modules (bold) overlaying the class structure (regular). Note how class dependencies across module boundaries determine module dependencies.

Directory structure



Declaring Modules

```
module monitor.observer {
 exports monitor.observer;
}

module monitor.observer.alpha {
 requires monitor.observer;
 exports monitor.observer.alpha;
}

module monitor.observer.beta {
 requires monitor.observer;
 exports monitor.observer.beta;
}

module monitor.statistics {
 requires monitor.observer;
 exports monitor.statistics;
}

module monitor.persistence {
 requires monitor.statistics;
 requires hibernate.jpa;
 exports monitor.persistence;
 exports monitor.persistence.entity;
}
```

```
module monitor.rest {
 requires spark.core;
 requires monitor.statistics;
 exports monitor.rest;
}

module monitor {
 requires monitor.observer;
 requires monitor.observer.alpha;
 requires monitor.observer.beta;
 requires monitor.statistics;
 requires monitor.persistence;
 requires monitor.rest;
}
```

Application Module Graph

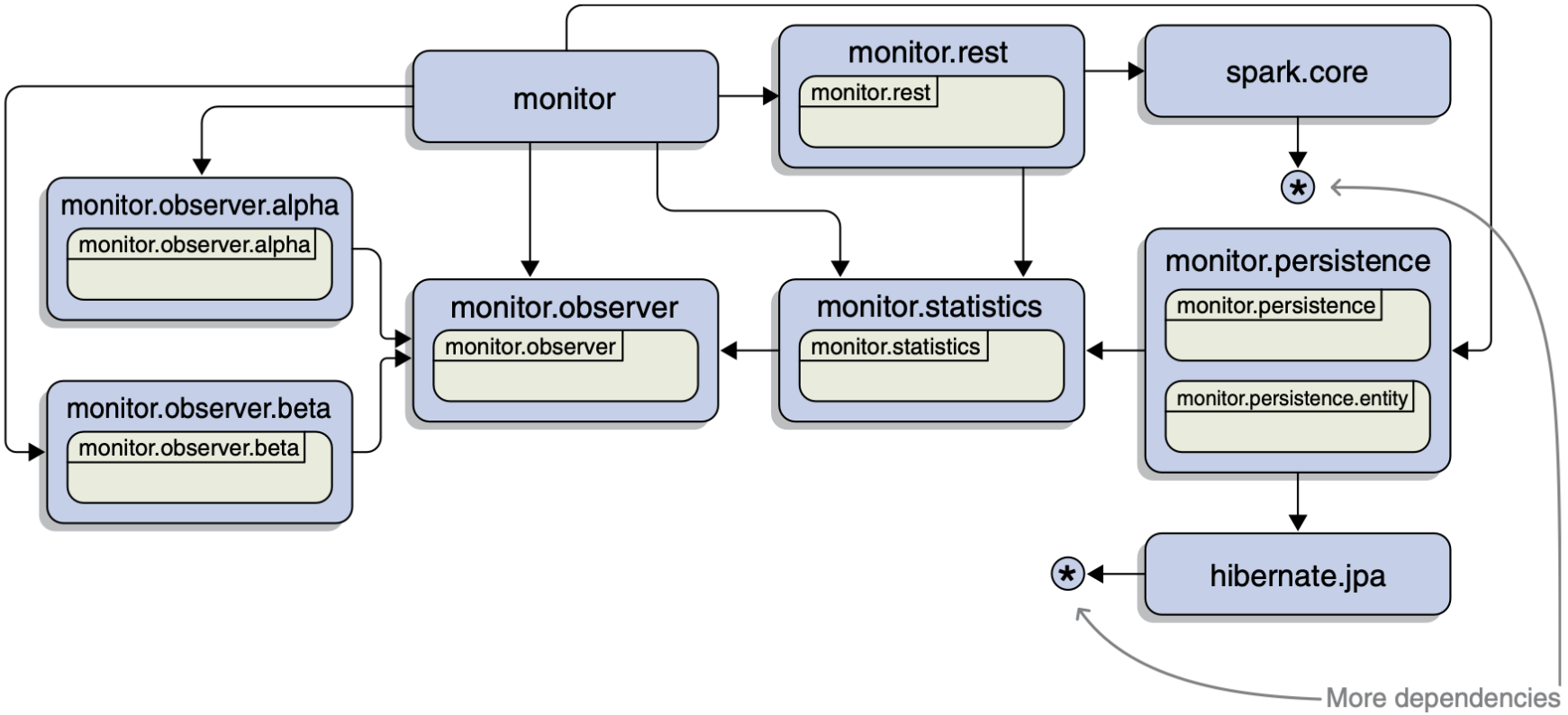
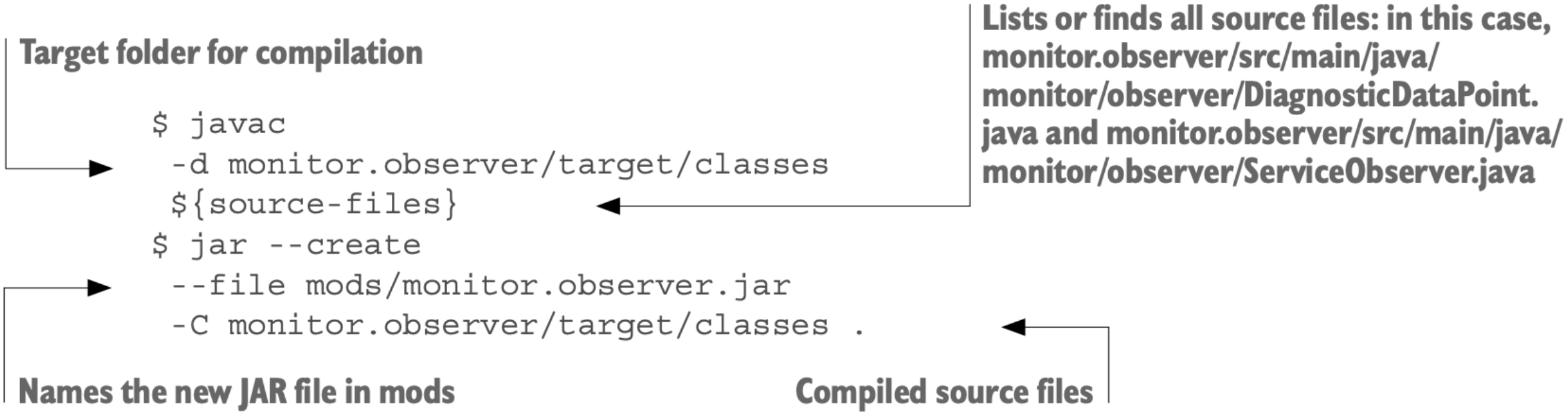


Figure 2.5 The application's module graph, showing modules with their exported packages and the dependencies between them. Unlike figure 2.3, this is not merely an architecture diagram; it's how the module system sees the application.

Compling and packaging



Running Application

```
$ java
 --module-path mods:libs
 --module monitor
```

Folders in which java searches for modules

Name of the module to launch

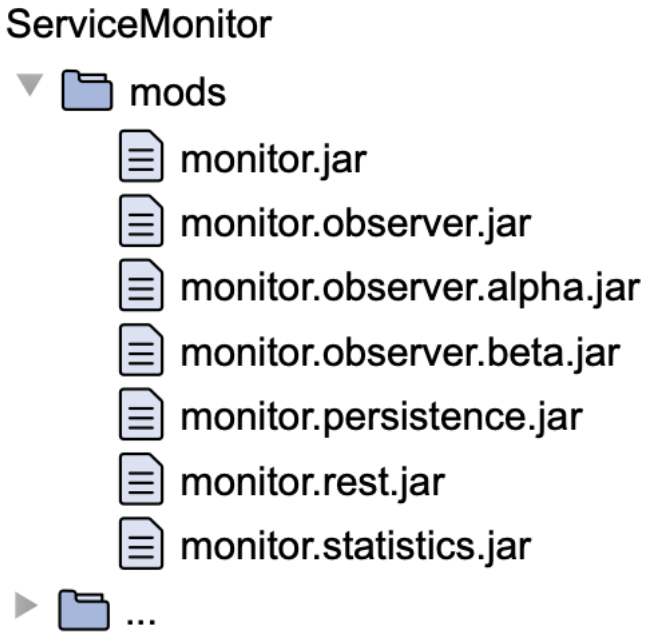
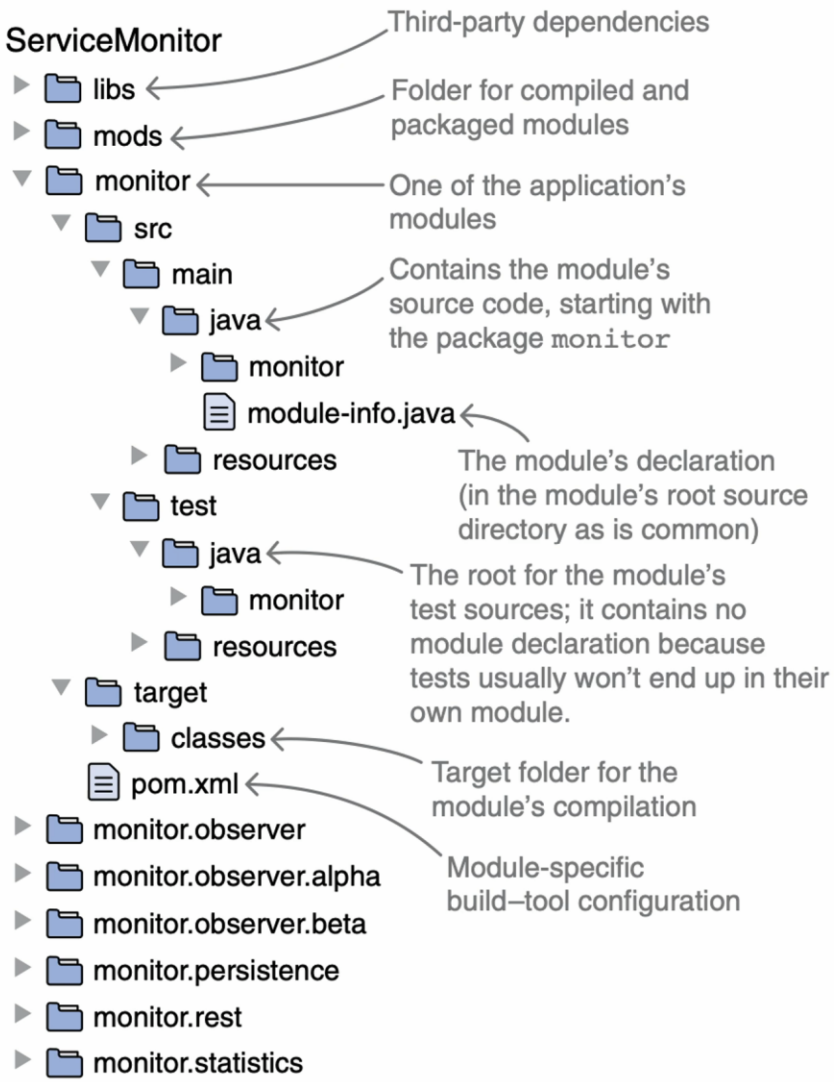


Figure 2.6 All the application modules, compiled and packaged in `mods` and ready to be launched.

Directory structure



Summary

- When you're modularizing an application, you can deduce the module dependencies from type dependencies across module boundaries. This makes creating an initial module-dependency graph a straightforward procedure.
- The directory structure of a multimodule project can be similar to what it would have been before Java 9, so existing tools and approaches will continue to work.
- The module declaration—a `module-info.java` file in the project's root directory—is the most obvious change that the module system brings to coding. It names the module and declares dependencies as well as the public API. Other than that, the way code is written is virtually unchanged.
- The commands `javac`, `jar`, and `java` have been updated to work with modules. The most obvious and relevant change is the module path (command-line option `--module-path` or `-p`). It parallels the class path but is used for modules.